



Instituto Tecnológico de Costa Rica
Campus Tecnológico Central Cartago
Escuela de Ingeniería en Computación

Proyecto 01: Restaurantes e2

Cesar Ricardo Gamboa Naranjo - 2024138712

Jose Daniel Morera Elizondo - 2024114502

IC4302-Bases de Datos II
Prof. Kenneth Obando Rodriguez
07 de mayo 2026

Introducción	3
Objetivo	3
Arquitectura	4
Física	4
Lógica	5
Servicios	6
API Backend	6
Nginx	6
Elasticsearch	7
MongoDB	10
PostgreSQL	13
Redis	13
Keycloak	14
CI/CD	15
Pruebas	16
Unitarias	16
Flujo de datos	17
Cliente-Reservación	18
Cliente-Búsqueda	18
Admin-Menu	19
Pasos para ejecución local	20
Docker compose	20
Kubernetes	20
Conclusiones	21
Link de Github	21
Referencias	22

Links

Github

<https://github.com/Daniel-M/Restaurante-DB-I-Proyecto.git>

Drive (Grabación)

Introducción

Este proyecto implementa una arquitectura distribuida basada en servicios para gestionar usuarios, autenticación, reservas de mesas y menús en restaurantes. Construye sobre la API REST desarrollada en la fase anterior (Tarea Corta 01).

Objetivo

Esta etapa tiene como objetivo implementar una arquitectura profesional sobre un sistema de restaurante inteligente que debe permitir la gestión de usuarios, restaurantes, menús, productos, reservas y pedidos, utilizando una arquitectura escalable y configurable. Además se espera que contemple tecnologías como CI/CD, replicación y particionado (sharding) de datos en MongoDB, un servidor de búsqueda con Elasticsearch, un balanceador de carga, cacheo con Redis, pruebas automáticas, escalabilidad de servicios y documentación detallada como si fuera una entrega a cliente final. Además, el sistema debe ser capaz de operar tanto con PostgreSQL como con MongoDB, permitiendo su configuración dinámica sin cambiar el código fuente. La siguiente tabla muestra las tecnologías en las cuales se llevó a cabo el proyecto.

Tecnología	Uso en el proyecto
FastAPI	API principal y microservicio de búsqueda
PostgreSQL	Base de datos relacional
MongoDB	Base NoSQL con replicación y sharding
Redis	Caché de respuestas frecuentes
ElasticSearch	Búsqueda textual de productos
Keycloak	Autenticación y autorización con JWT
Nginx	Balanceador de carga y reverse proxy
Docker Compose	Orquestación local
Kubernetes	Escalabilidad horizontal
GitHub Actions	Pipeline CI/CD

Tabla 1. Tecnologías utilizadas.

Arquitectura

En esta sección se describen los modelos de arquitectura tanto física como lógica, además se presentan diagramas para mejorar el entendimiento de estos mismo.

Física

El diagrama de arquitectura física representa la infraestructura real del sistema y la manera en que los distintos servicios se encuentran desplegados y conectados mediante contenedores Docker, siendo así, cada bloque representa un contenedor o servicio desplegado dentro del entorno Docker. El cliente accede al sistema a través de Nginx, el cual distribuye las solicitudes hacia el API Backend y los servicios especializados. El backend se comunica con PostgreSQL o MongoDB para el almacenamiento de datos, Redis para caché, Elasticsearch para búsquedas avanzadas y Keycloak para autenticación y autorización. Además, se utilizan volúmenes persistentes para garantizar la conservación de la información.

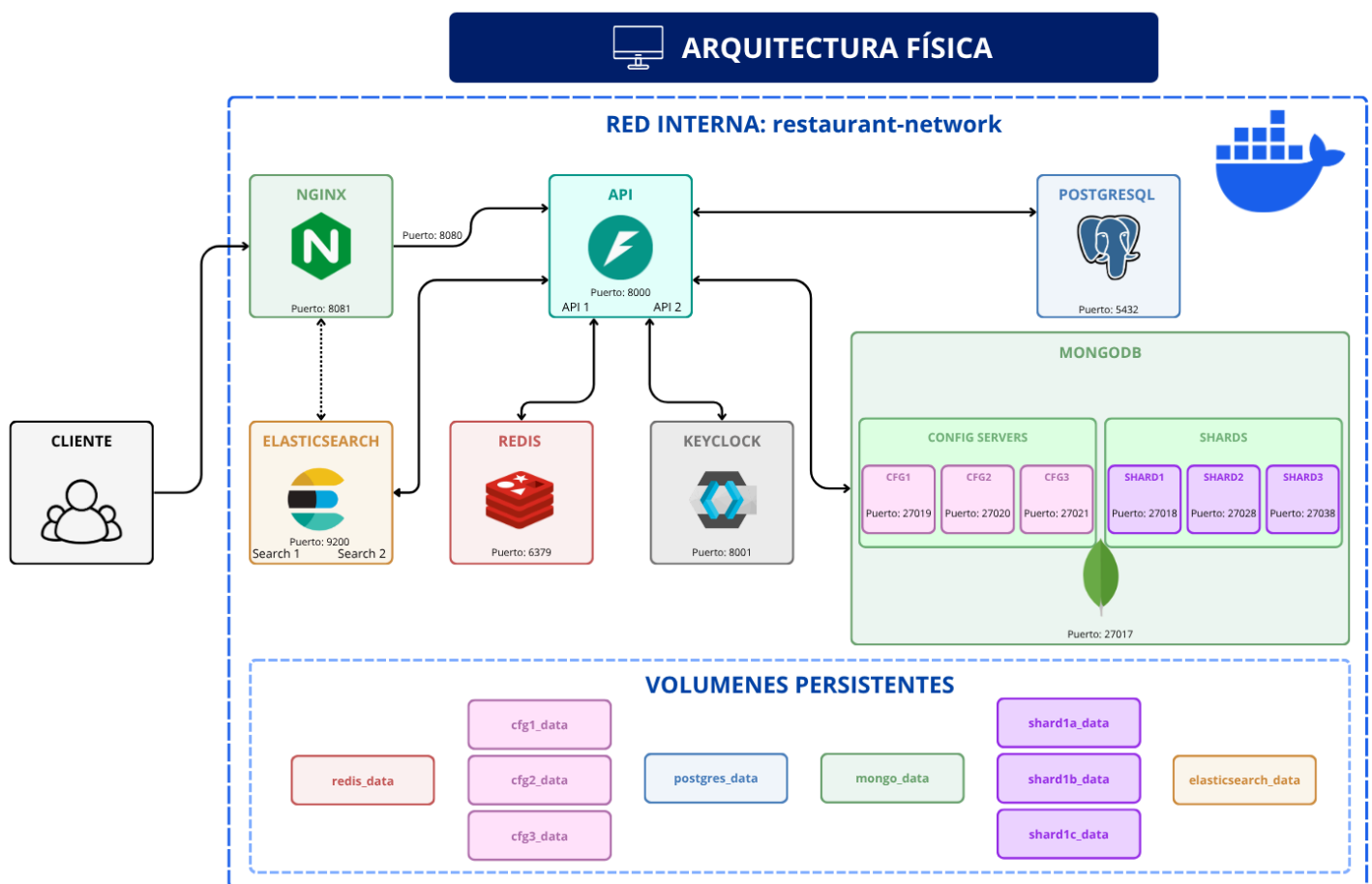


Fig 1. Arquitectura física.

Lógica

El diagrama de arquitectura lógica representa la organización conceptual del sistema y la distribución de responsabilidades entre sus componentes. La aplicación se divide en capas que incluyen presentación, lógica de negocio, acceso a datos y servicios externos. El backend centraliza funcionalidades como autenticación, gestión de usuarios, reservaciones, órdenes y menús, mientras que PostgreSQL, MongoDB, Redis, Elasticsearch y Keycloak complementan el funcionamiento general del sistema.

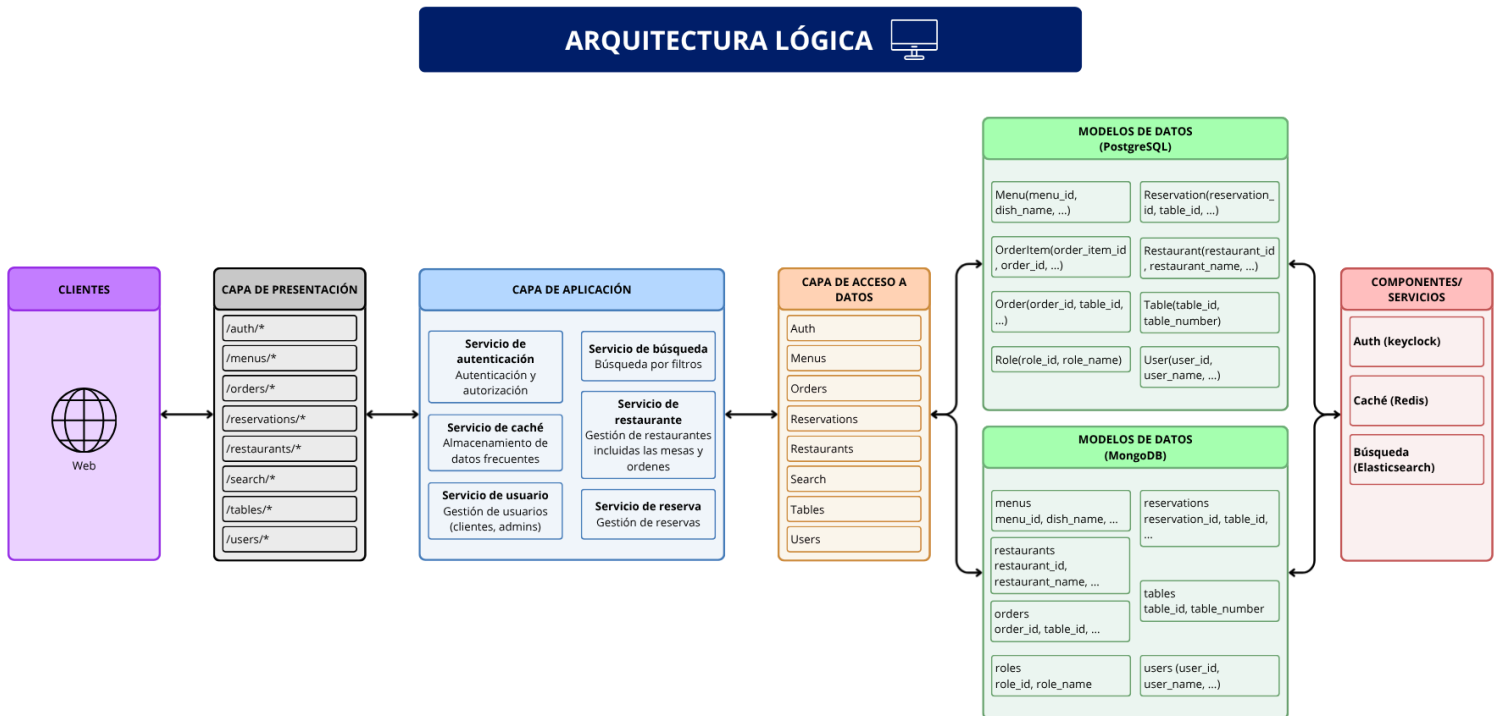


Fig 2. Arquitectura lógica

Servicios

Este apartado trata cada uno de los microservicios del proyecto y se explican las responsabilidades de cada uno.

API Backend

Fast API

Puerto: interno-80, externo-8000

Ubicación: main.py

Responsabilidades:

Autenticación & Autorización - Gestión de usuarios y roles con Keycloak

Gestión de Menús - Crear, actualizar y consultar menús y platillos

Gestión de Órdenes - Crear órdenes, ver órdenes

Gestión de Reservaciones - Reservar mesas, cancelar reservas

Gestión de Restaurantes - CRUD de restaurantes

Gestión de Mesas - Configurar mesas disponibles por restaurante

Gestión de Usuarios - Información y creación de clientes

Routers:

/routers/auth.py - Login, registro, validación de tokens

/routers/menus.py - Operaciones de menús

/routers/orders.py - Operaciones de órdenes

/routers/reservations.py - Operaciones de reservaciones

/routers/restaurants.py - Operaciones de restaurantes

/routers/tables.py - Operaciones de mesas

/routers/users.py - Operaciones de usuarios

Nginx

Reverse Proxy

Puerto: interno-80, externo-8080

Ubicación: /nginx/default.conf

Responsabilidades:

Enrutamiento de Tráfico - Distribuye peticiones entre microservicios

Balanceo de Carga

- Enruta /api/ al microservicio API
- Enruta /search/ al microservicio Elasticsearch

Rutas:

http://localhost:8080/api/** → Microservicio API

http://localhost:8080/search/** → Microservicio de búsqueda

http://localhost:8081/search/** → Microservicio de búsqueda(en el balanceador de kubernetes)



Fig x. Dirección servicio API.

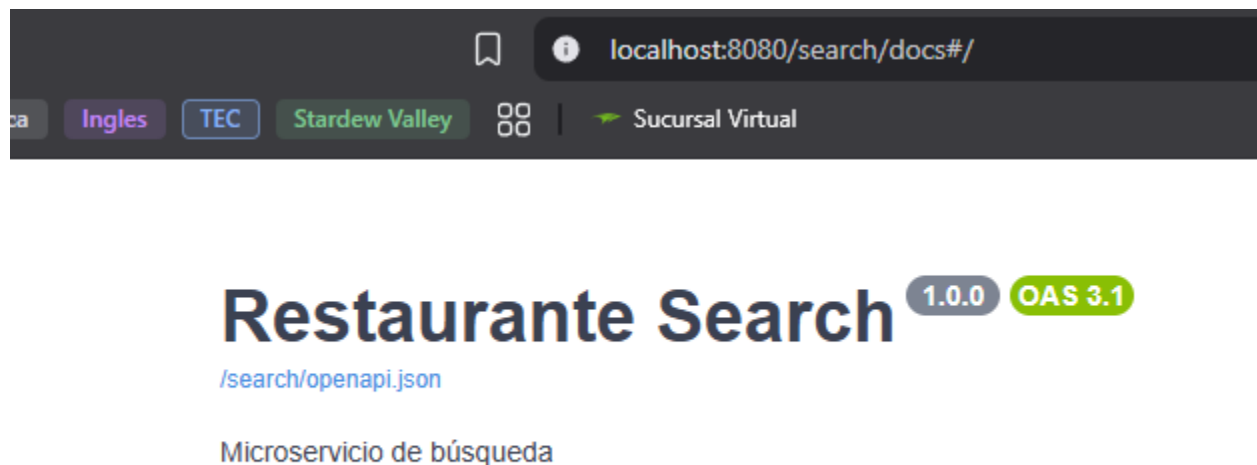


Fig x. Dirección servicio search.

Elasticsearch

Búsqueda avanzada

Puerto: Elasticsearch nativo-9200, Microservicio de búsqueda-8002

Ubicación: /app/search

Responsabilidades:

Crear Índice - Define estructura de datos para búsqueda

Indexación de Documentos - Indexa menús y platillos en formato JSON para búsqueda rápida
Búsqueda - Búsqueda full-text en múltiples campos

- Búsqueda por nombre de plato
- Búsqueda por categoría

Actualizar Índice - Sincroniza índice para disponibilidad inmediata de nuevos datos.

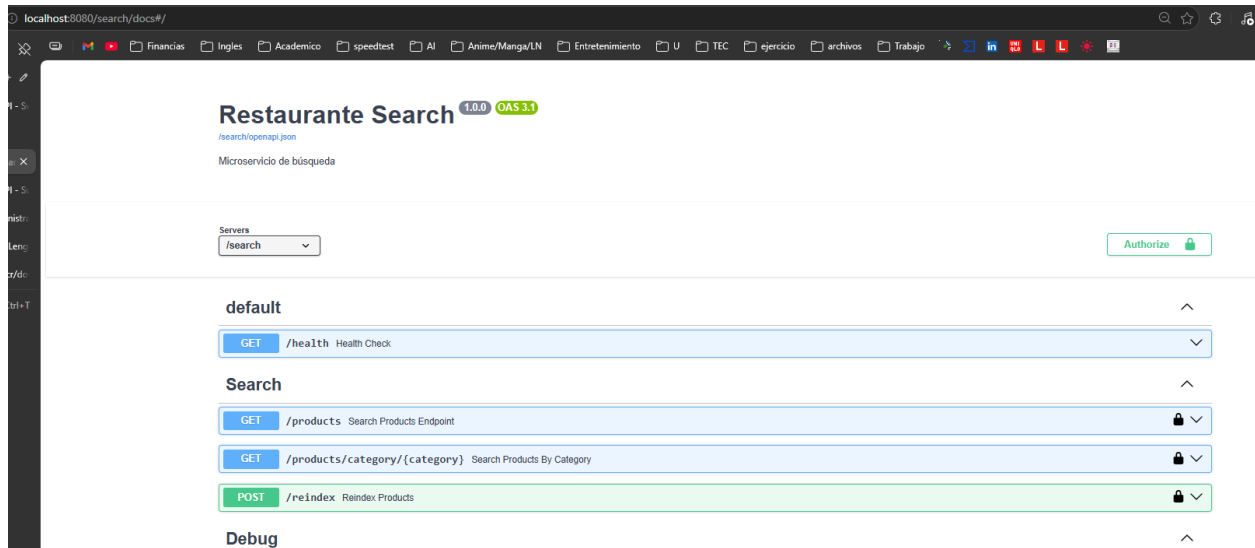


Fig 3. Endpoints search.

Archivos relacionados:

/search/elasticsearch_client.py - Cliente de conexión

/search/search_service.py - Lógica de búsqueda

/routers/search.py - Implementación endpoints

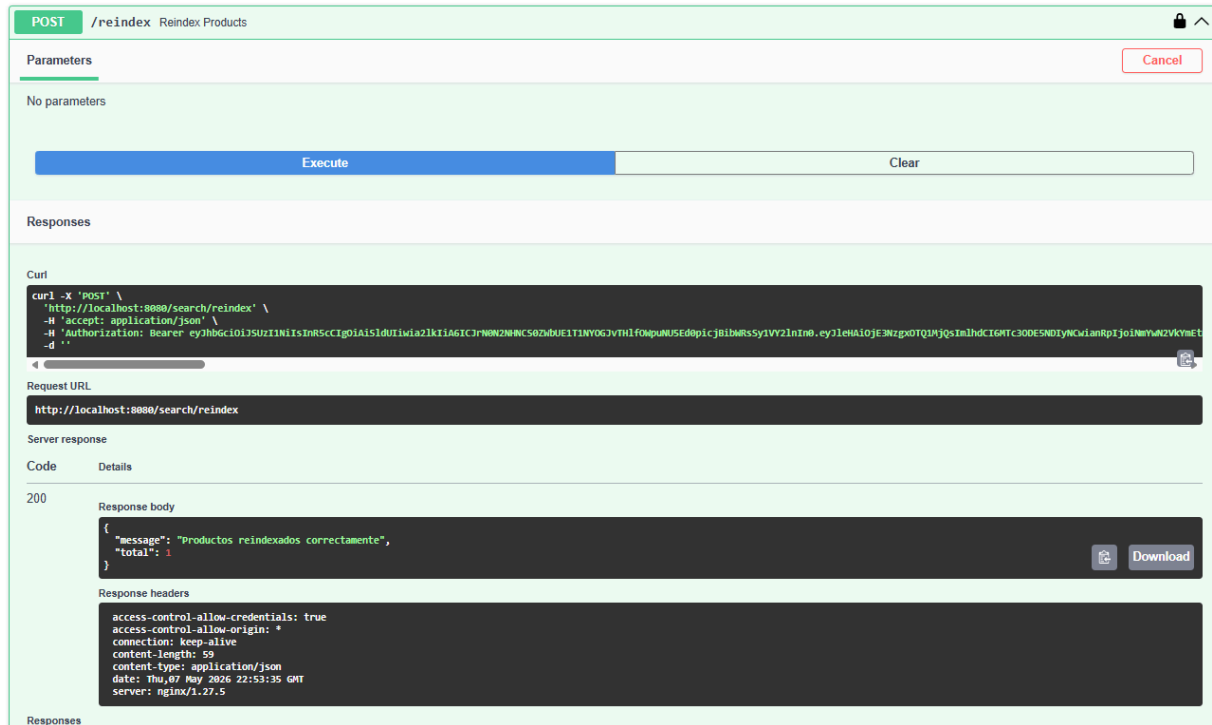


Fig 4. Ejecución de reindex.

En la figura 4 podemos ver como al ejecutar el endpoint este accede a los datos que hay cargados para poder tenerlos a disponibilidad de la búsqueda para los endpoints posteriores.

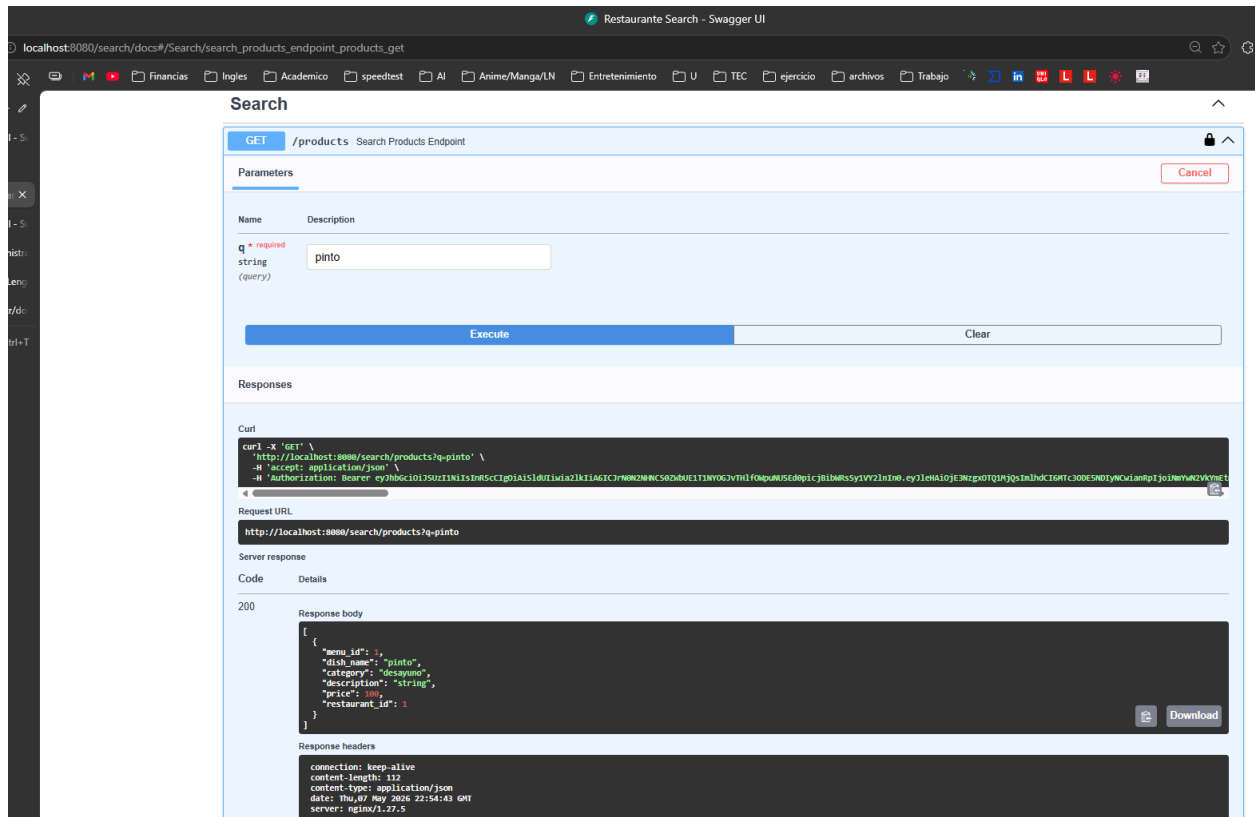


Fig 5. Búsqueda por nombre.

En la figura 5 podemos ver cómo al ingresar el nombre de un platillo este devuelve la información del platillo.

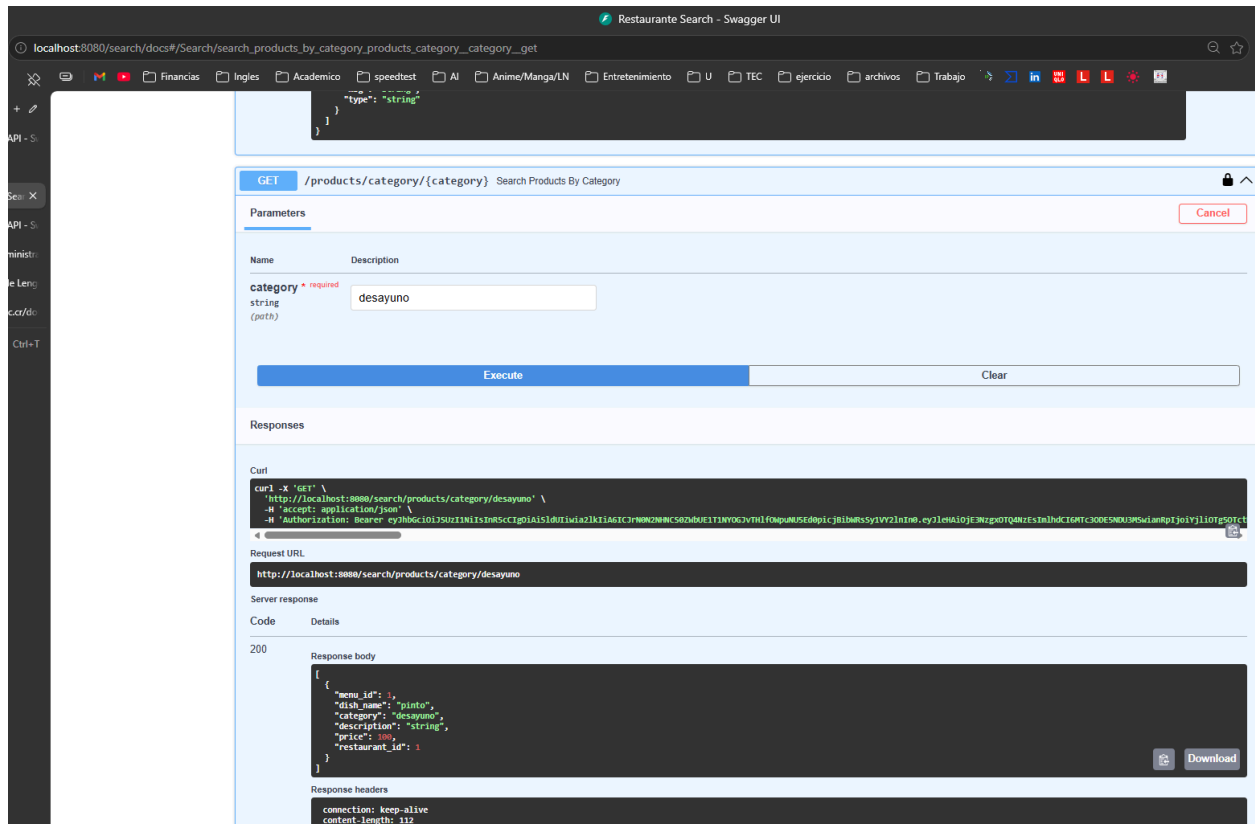


Fig 6. Búsqueda por categoría.

En la figura 6 podemos ver cómo al buscar una categoría de comida este endpoint nos muestra todos los menús con esa categoría.

MongoDB

Base de Datos NoSQL

Puerto: 27017 (mongos router)

Ubicación: /dbs/mongo

Responsabilidades:

Almacenamiento de Datos - Documentos JSON de restaurantes, menús, órdenes, reservaciones

Replicación - Alta disponibilidad con replica sets

Sharding - Distribución horizontal de datos para escalabilidad

Arquitectura:

Config Server Replica Set (cfgReplSet) - Metadatos del clúster (3 nodos)

Shard 1 (shard1ReplSet) - Datos particionados (3 nodos: shard1a, shard1b, shard1c)

Router (mongos) - Punto de conexión único para la API

A continuación se presentara una serie de comando en donde se muestra la reacción de los sharding durante diversas situaciones.

```
(.venv) PS C:\Users\Daniel\OneDrive - Estudiantes ITCR\@TEC\V Semestre\Bases II\proyecto I\Restaurante-DB-I-Proy  
ecto> docker exec -it restaurant-shard1a mongosh --port 27018 --quiet --eval "rs.status().members.map(m=>({name:m.n  
ame,state:m.stateStr,health:m.health}))"  
[  
  { name: 'shard1a:27018', state: 'PRIMARY', health: 1 },  
  { name: 'shard1b:27018', state: 'SECONDARY', health: 1 },  
  { name: 'shard1c:27018', state: 'SECONDARY', health: 1 }  
]
```

Fig 7. Shards con primary.

En la figura 7 podemos ver a los 3 shardings levantados y cual es el principal.

```
(.venv) docker exec -it restaurant-cfg1 mongosh --port 27019 --quiet --eval "rs.status().members.map(m=>({name:m.n  
ame,state:m.stateStr}))"  
[  
  { name: 'cfg1:27019', state: 'PRIMARY' },  
  { name: 'cfg2:27019', state: 'SECONDARY' },  
  { name: 'cfg3:27019', state: 'SECONDARY' }  
]
```

Fig 8. Replicas con primary.

en la figura 8 podemos ver a las 3 réplicas junto con su primary.

```
(.venv) PS C:\Users\Daniel\OneDrive - Estudiantes ITCR\@TEC\V Semestre\Bases II\proyecto I\Restaurante-DB-I-Proy  
ecto> docker exec -it restaurant-mongos mongosh --port 27017 --quiet --eval "db.getSiblingDB('restaurant_mongo_d  
b').reservations.insertOne({reservation_id: 10001, client_id: 1, table_id: 1, reservation_date: new Date(), stat  
us: 'terminal-test-before-failure'})"  
{  
  acknowledged: true,  
  insertedId: ObjectId('69fd1b715553d7068344ba89')  
}
```

Fig 9. Insertar dato en mongo.

En la figura 9 se ejecuta un comando para insertar una reservación en la tabla y así exponer a más estrés a los sharding más adelante.

```
(.venv) PS C:\Users\Daniel\OneDrive - Estudiantes ITCR\@TEC\V Semestre\Bases II\proyecto I\Restaurante-DB-I-Proy  
ecto> docker stop restaurant-shard1a  
restaurant-shard1a  
(.venv) PS C:\Users\Daniel\OneDrive - Estudiantes ITCR\@TEC\V Semestre\Bases II\proyecto I\Restaurante-DB-I-Proy
```

Fig 10. Apagar el shard primary.

En la figura 10 bajamos al sharding principal para ver como reacciona el codigo.

```
(.venv) PS C:\Users\Daniel\OneDrive - Estudiantes ITCR\@TEC\V Semestre\Bases II\proyecto I\Restaurante-DB-I-Proy
ecto> docker exec -it restaurant-shard1b mongosh --port 27018 --quiet --eval "rs.status().members.map(m=>({name:
m.name,state:m.stateStr,health:m.health}))"
[
  {
    name: 'shard1a:27018',
    state: '(not reachable/healthy)',
    health: 0
  },
  { name: 'shard1b:27018', state: 'PRIMARY', health: 1 },
  { name: 'shard1c:27018', state: 'SECONDARY', health: 1 }
]
```

Fig 11. Nuevo shard primary.

En la figura 11 podemos ver cómo una vez que el principal se da de baja el programa selección a otro como su primario, si no hubiese sido el principal simplemente lo hubiera dado de baja.

```
(.venv) PS C:\Users\Daniel\OneDrive - Estudiantes ITCR\@TEC\V Semestre\Bases II\proyecto I\Restaurante-DB-I-Proy
ecto> docker start restaurant-shard1a
restaurant-shard1a
```

Fig 12. Subir shard caído.

En la figura 12 se ejecuta el comando para volver a levantar el shard que antes fue primario.

```
(.venv) PS C:\Users\Daniel\OneDrive - Estudiantes ITCR\@TEC\V Semestre\Bases II\proyecto I\Restaurante-DB-I-Proy
ecto> docker exec -it restaurant-shard1b mongosh --port 27018 --quiet --eval "rs.status().members.map(m=>({name:
m.name,state:m.stateStr,health:m.health}))"
[
  { name: 'shard1a:27018', state: 'SECONDARY', health: 1 },
  { name: 'shard1b:27018', state: 'PRIMARY', health: 1 },
  { name: 'shard1c:27018', state: 'SECONDARY', health: 1 }
]
```

Fig 13. Todos los shards arriba pero con el nuevo primary.

En la figura 13 se ejecuta un comando para poder ver a todos los shards disponibles, ya sean que estén vivos o no.

```
(.venv) PS C:\Users\Daniel\OneDrive - Estudiantes ITCR\@TEC\V Semestre\Bases II\proyecto I\Restaurante-DB-I-Proy
ecto> docker exec -it restaurant-mongos mongosh --port 27017 --quiet --eval "db.getSiblingDB('restaurant_mongo_d
b').reservations.find({reservation_id: {$in: [10001]}}).pretty()"
[
  {
    _id: ObjectId('69fd1b715553d7068344ba89'),
    reservation_id: 10001,
    client_id: 1,
    table_id: 1,
    reservation_date: ISODate('2026-05-07T23:08:33.306Z'),
    status: 'terminal-test-before-failure'
  }
]
```

Fig.14. Persistencias de los datos.

En la figura 12 se ejecuta un comando para mostrar la información de la reserva anteriormente hecha para que así quede demostrado la persistencia de los datos a pesar de que se cayera el principal.

Colecciones:

restaurants

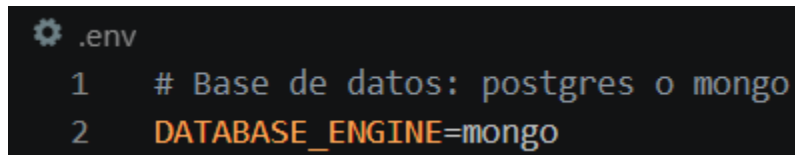
menus

orders

reservations

tables

Users



```
1 # Base de datos: postgres o mongo
2 DATABASE_ENGINE=mongo
```

Fig.15. Opción de MongoDB.

En la figura 15 se puede ver la forma de configurar el motor de búsqueda con el que se va ejecutar, en este caso se trabaja con mongo.

PostgreSQL

Base de Datos Relacional

Puerto: 5432

Ubicación: /dbs/postgres

Responsabilidades:

Almacenamiento Estructurado - Datos con relaciones complejas

Integridad Referencial - Constraints y foreign keys

Transacciones ACID - Garantía de consistencia

Tablas:

Roles

Users

Restaurants

Menus

Tables

Orders

Order_Items

Reservations

```

. .env
1  # Base de datos: postgres o mongo
2  DATABASE_ENGINE=postgres

```

Fig 16. Opción de PostgreSQL

En la figura 16 se puede ver la forma de configurar el motor de búsqueda con el que se va ejecutar, en este caso se trabaja con Postgres.

Redis

Caché Distribuido

Puerto: 6379

Ubicación: /app/cache

Responsabilidades:

Caché en Memoria - Almacena datos frecuentemente accedidos

Time to live - tiempo para que el caché viva

Archivos relacionados:

redis_client.py - Cliente de conexión

cache_service.py - Lógica de caché

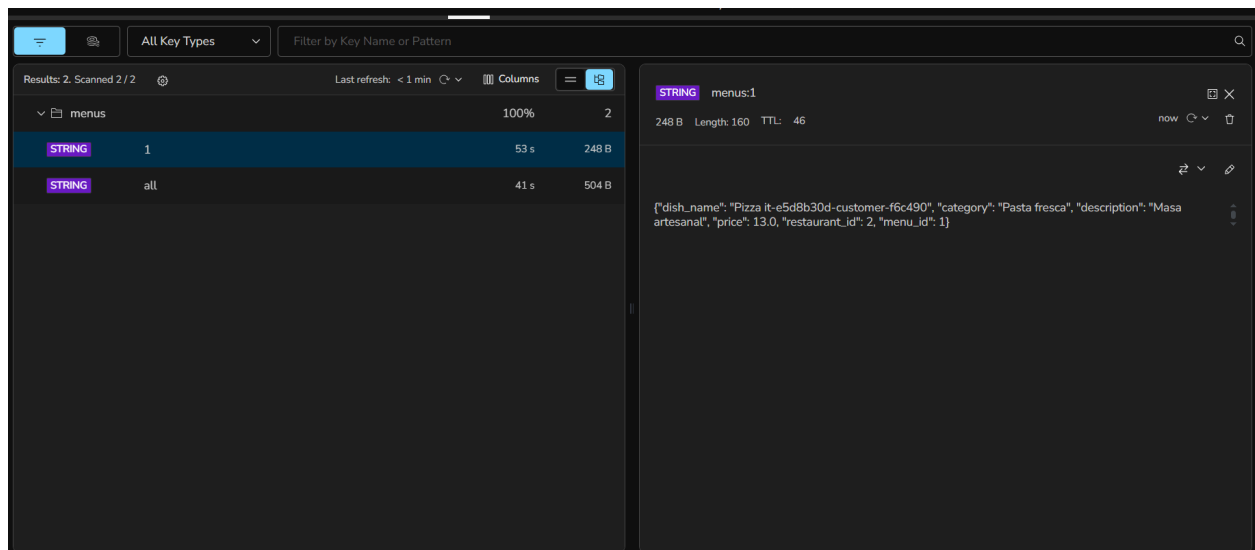


Fig 17. Vista en redis insight de las llaves creadas del caché.

Keycloak

Autenticación y Autorización

Puerto: 8001

Ubicación: autenticador

Responsabilidades:

Gestión de Roles - Asignación de permisos (admin, cliente)

Token JWT - Emisión y validación de tokens de sesión

Gestión de Usuarios - Registro y sesión de usuarios

Archivos relacionados:

keycloak_login.py - Lógica de login

keycloak_register_client.py - Registro de clientes

keycloak_register_admin.py - Registro de administradores

keycloak_validation.py - Validación de tokens

keycloak_dependencies.py - Dependencias FastAPI

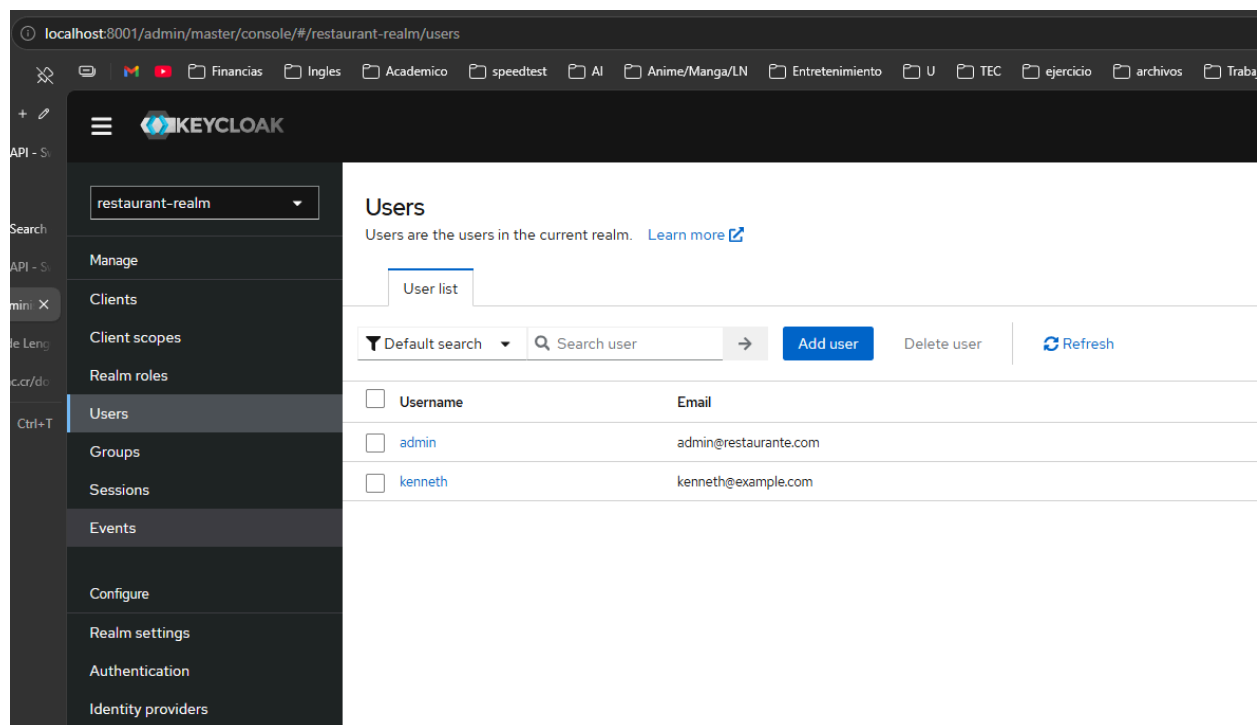


Fig 18. Keycloak clientes.

CI/CD

Continuous Integration/ Continuous Delivery

El proyecto utiliza GitHub Actions como herramienta de integración y entrega continua. El pipeline se ejecuta automáticamente al realizar cambios en el repositorio y se encarga de instalar dependencias, ejecutar pruebas unitarias, ejecutar pruebas de integración, construir la imagen Docker de la aplicación y publicarla en el Container Registry del repositorio.

Etapas:

- Checkout
- Setup Python
- Install dependencies
- Unit tests
- Integration tests
- Docker build
- Push image

A continuación se muestran dos figuras que demuestran la correcta funcionalidad de este apartado de CI/CD.

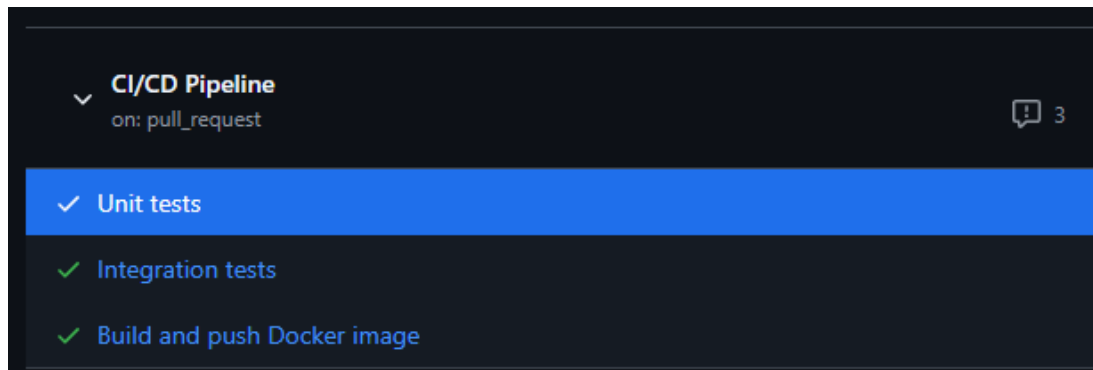


Fig 19. Pipeline exitoso.

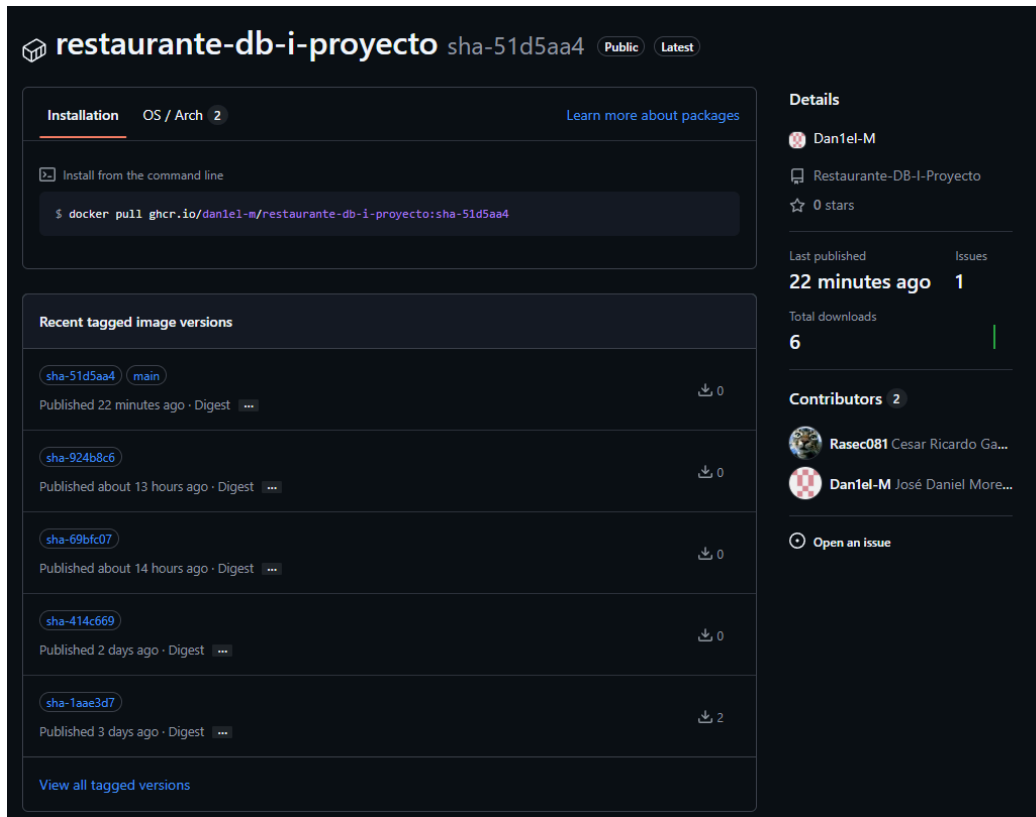


Fig 20. Imagen publicada en GHCR.

Archivos relacionados:

.github/workflows/ci-cd.yml - Configuración

Pruebas

Para las pruebas se separa en dos secciones distintas, las pruebas unitarias y las pruebas de integración.

Unitarias

Las pruebas unitarias se basan en probar cada uno de los componentes del proyecto por separado, y como se ve en la figura de esta sección el coverage alcanzó un 94,56%.

Se prueba con el siguiente comando:

```
python -m pytest -q backend/tests/unit
```

```
----- coverage: platform win32, python 3.11.9-final-0 -----
Name                               Stmts  Miss  Cover   Missing
-----
backend\app\routers\debug.py         7      1   86%    11
backend\app\routers\menus.py        74      6   92%   44-45, 115, 119-120, 152
backend\app\routers\restaurants.py  43      4   91%    79, 85, 92, 110
backend\app\routers\tables.py       49      1   98%    84
backend\app\routers\users.py        59      7   88%   55, 62, 93-94, 115, 138-139
backend\dao.py                      227    16   93%   30, 33, 36, 39, 42, 85-90, 93, 114, 129, 149, 173
backend\models\menus.py              16      1   94%    27
backend\models\order_items.py        14      1   93%    20
backend\models\orders.py             16      1   94%    22
backend\models\reservations.py       14      1   93%    20
backend\models\roles.py              10      1   90%    16
backend\models\tables.py             15      1   93%    26
-----
TOTAL                               754    41   95%

15 files skipped due to complete coverage.
Coverage HTML written to dir htmlcov
Coverage XML written to file coverage.xml

Required test coverage of 90% reached. Total coverage: 94.56%
```

Fig 21. Coverage pruebas unitarias.

Integración

En cambio las pruebas de integración prueban como tal, la comunicación de unos servicios con otros, es decir, funcionamiento en conjunto de todo el proyecto.

Las pruebas unitarias se corren con el siguiente comando:

```
python -m pytest -q backend/tests/integration
```

```
(.venv) PS C:\Users\Daniel\OneDrive - Estudiantes ITCR\@TEC\V Semestre\Bases II\proyecto I\Restaurante-DB-I-Proyecto> python -m pytest -q backend/tests/integration
----- coverage: platform win32, python 3.12.10-final-0 -----
Name                               Stmts  Miss  Cover   Missing
-----
backend\app\routers\menus.py        74      12    84%    23-24, 44-45, 51, 75, 102, 109, 119-120, 146, 152
backend\app\routers\orders.py        21       1    95%      37
backend\app\routers\restaurants.py   43       4    91%    27, 72, 92, 110
backend\app\routers\tables.py        49       8    84%    25, 37, 43, 63, 77-78, 84, 102
backend\app\routers\users.py         59       8    86%    28, 35, 55, 93-94, 115, 138-139
backend\dao.py                       254      23    91%    30, 33, 36, 39, 42, 99-107, 110, 128-129, 132, 138, 156-157, 160, 180, 203, 215, 273-274
backend\models\order_items.py        14       1    93%      20
backend\models\orders.py             16       1    94%      22
backend\models\reservations.py       14       1    93%      20
backend\models\roles.py              10       1    90%      16
backend\models\tables.py             15       1    93%      26
backend\models\users.py              15       1    93%      21
-----
TOTAL                                792      62    92%

16 files skipped due to complete coverage.
Coverage HTML written to dir htmlcov
Coverage XML written to file coverage.xml

Required test coverage of 90% reached. Total coverage: 92.17%
```

Fig 22. Coverage pruebas de integración.

Flujo de datos

En este apartado se detallarán por medio de diagramas el flujo en tres casos de uso de la aplicación, uno es el caso de uso de un cliente queriendo pedir una reservación de un restaurante y el siguiente es el caso de uso de un cliente haciendo una búsqueda de los productos disponibles, y por último el del administrador agregando platillos al menú, hipotéticamente el cliente estará conectado desde una base de datos con motor PostgreSQL, y el admin estará utilizando un motor MongoDB. En cada diagrama se explicará brevemente el funcionamiento y su interacción con los demás componentes para completar las acciones mencionadas.

Cliente-Reservación

Este diagrama representa el proceso de reservación realizado por un cliente. La solicitud es enviada desde la interfaz web hacia el backend, donde se valida la autenticación del usuario y la disponibilidad de la reservación antes de almacenar la información en la base de datos y retornar la respuesta correspondiente.

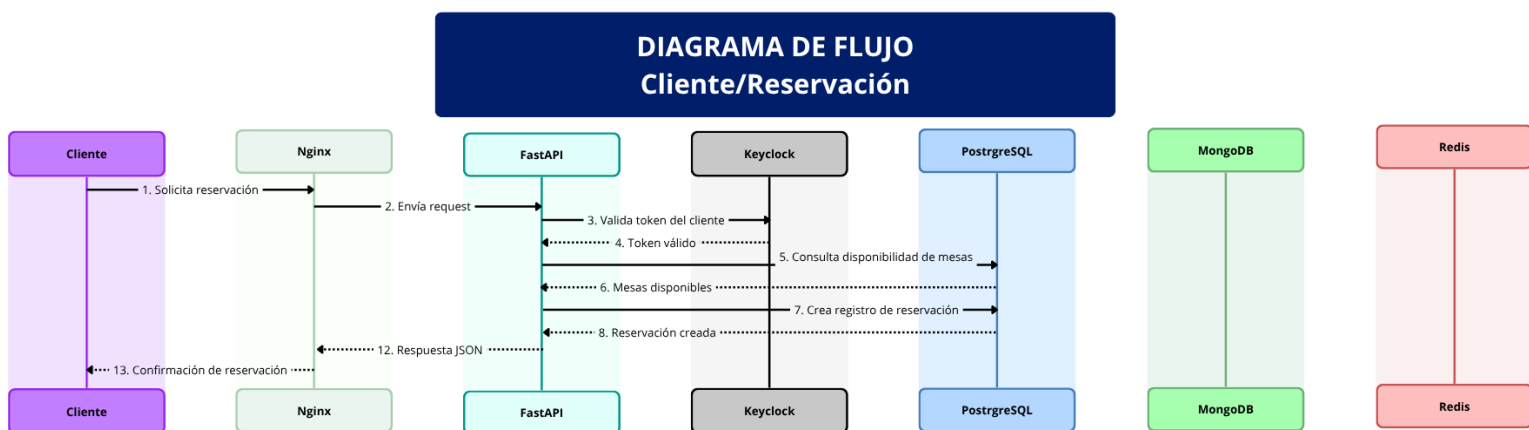


Fig 23. Diagrama de flujo cliente/reservación

Cliente-Búsqueda

El flujo Cliente-Búsqueda muestra cómo el sistema procesa búsquedas avanzadas realizadas por los usuarios. El backend primero verifica si la información existe en Redis y, en caso contrario, consulta Elasticsearch para obtener resultados relacionados con productos o menús, devolviendo posteriormente la información al cliente.

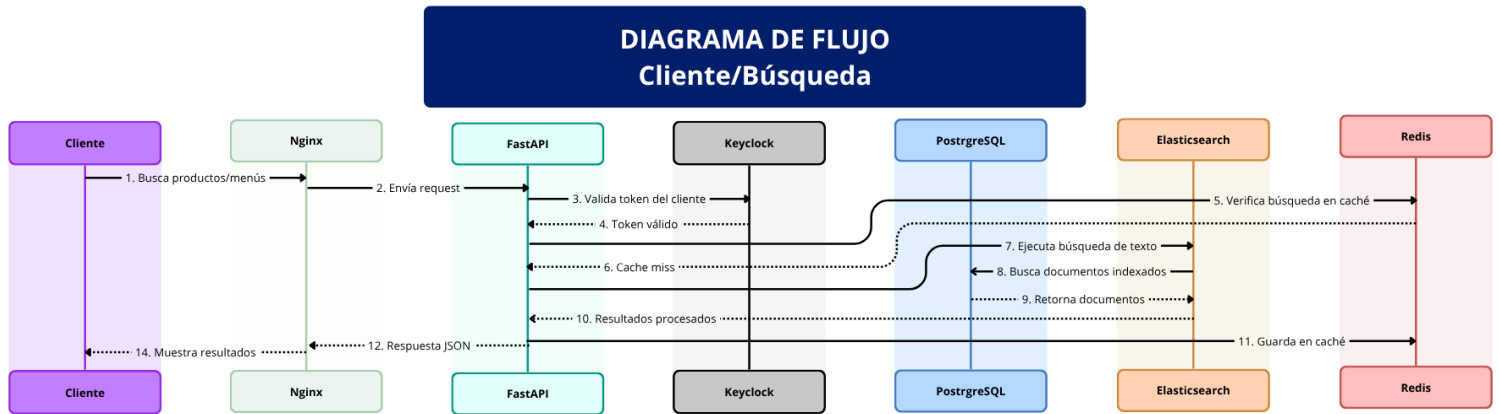


Fig 24. Diagrama de flujo cliente/búsqueda

Admin-Menu

Este diagrama representa el proceso mediante el cual un administrador agrega o modifica platillos del menú. El backend valida los permisos administrativos utilizando Keycloak, almacena la información en MongoDB y posteriormente actualiza Elasticsearch para mantener sincronizados los mecanismos de búsqueda del sistema.

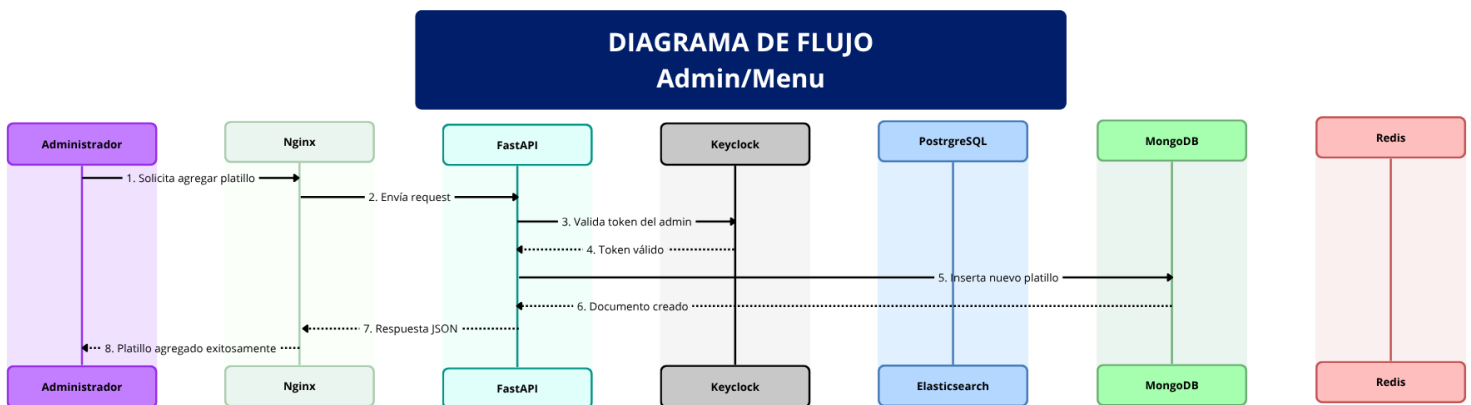


Fig 25. Diagrama de flujo admin/menu

Pasos para ejecución local

Este apartado menciona los pasos a seguir para lograr ejecutar el proyecto de una manera correcta, sin embargo en el [README.md](#) del repositorio contiene una explicación más detallada del proceso de instalación, configuración y ejecución.

Docker compose

Docker Compose permite levantar todos los servicios necesarios del sistema, incluyendo la API principal, el microservicio de búsqueda, las bases de datos, Redis, Keycloak, ElasticSearch y Nginx.

Para construir las imágenes y levantar los contenedores:

```
docker compose up -d --build
```

Para ejecutar el proyecto con escalabilidad horizontal en los servicios de API y búsqueda:

```
docker compose up -d --build --scale api=2 --scale search=2
```

Este comando levanta dos réplicas del servicio api y dos réplicas del servicio search, permitiendo validar que el balanceador de carga distribuya las solicitudes entre varias instancias.

The screenshot displays a web browser window with a light blue background. It shows the results of a curl command. The 'Curl' section contains the command: `curl -X 'GET' \ 'http://localhost:8080/api/debug/instance' \ -H 'accept: application/json'`. The 'Request URL' section shows `http://localhost:8080/api/debug/instance`. The 'Server response' section is divided into two parts: 'Code' and 'Details'. The 'Code' section shows a status code of 200. The 'Details' section is further divided into 'Response body' and 'Response headers'. The 'Response body' shows a JSON object: `{ "service": "api", "container": "d16533801e96" }`. The 'Response headers' section shows: `connection: keep-alive, content-length: 44, content-type: application/json, date: Fri, 08 May 2026 00:31:55 GMT, server: nginx/1.27.5`.

Fig 26. Primera conexión al microservicio.

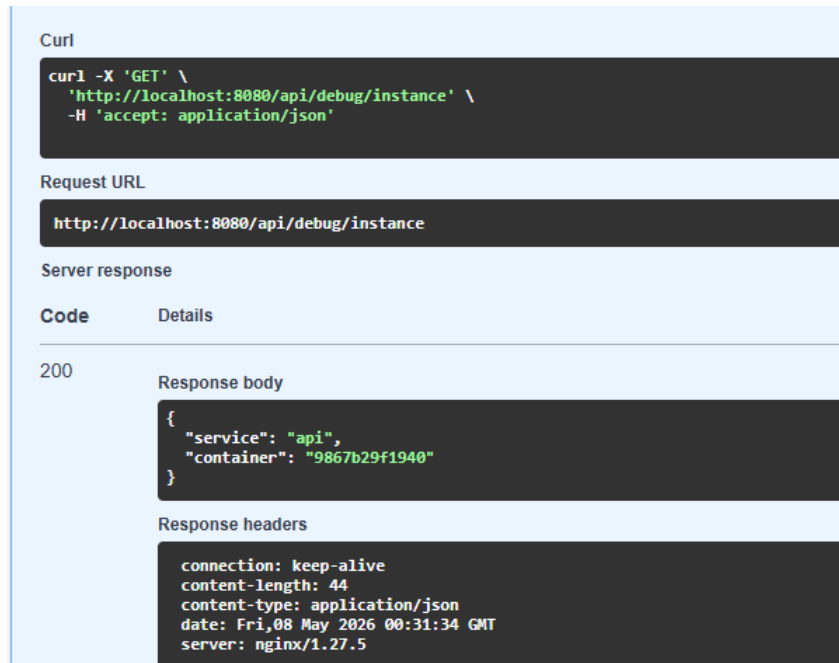


Fig x. Segunda conexión al microservicio, distinto container.

Para apagar los contenedores y eliminar los volúmenes asociados:

```
docker compose down -v
```

Para verificar que los contenedores estén en ejecución:

```
docker ps
```

Para revisar los logs de un servicio específico:

```
docker compose logs nombre_servicio
```

Kubernetes

El proyecto también puede ejecutarse localmente con Kubernetes utilizando Docker Desktop con Kubernetes habilitado. Esta configuración permite escalar los servicios principales modificando la cantidad de réplicas en los manifiestos o utilizando comandos de `kubectl scale`.

Para levantar el proyecto con el script automatizado:

Este script se encarga de construir la imagen local, crear el namespace restaurante, aplicar ConfigMaps, Secrets, Services, Deployments y ejecutar los trabajos de inicialización necesarios para levantar el entorno.

```
powershell -ExecutionPolicy Bypass -File kubernetes/_initK8s.ps1
```

Para eliminar todos los recursos del proyecto en Kubernetes:

Este comando elimina el namespace restaurante junto con todos los recursos asociados, incluyendo pods, services, deployments, jobs, ConfigMaps y Secrets creados para el proyecto.

```
kubectl delete namespace restaurante
```

Para verificar pods, servicios o deployments:

```
kubectl get pods -n restaurante  
kubectl get services -n restaurante  
kubectl get deployments -n restaurante
```

Conclusiones

Este proyecto ayudó mucho a conocer de bastantes herramientas que se utilizan de forma frecuente en el mundo laboral, dando una oportunidad para conocer nuevas herramientas y aprender de estas tecnologías, a través del desarrollo de esta API REST aprendimos a implementar una bases de datos distribuidas con MongoDB configurado con replicación y sharding, desarrollar microservicios escalables en el que utilizamos FastAPI, e integrar autenticación segura con Keycloak y JWT. La implementación de caché con Redis, búsqueda avanzada con Elasticsearch, y orquestación con Docker Compose nos mostró cómo construir sistemas de producción reales. Además, el uso de herramientas de inteligencia artificial permitió agilizar procesos de documentación, investigación y apoyo durante el desarrollo del proyecto.

Referencias

<https://fastapi.tiangolo.com/deployment/docker/#dockerfile>

<https://skycloak.io/blog/keycloak-fastapi-python-api-authentication/>

<https://github.com/imjoseangel/fastapi-keycloak/blob/devel/assets/Integrating%20FastAPI%20with%20Keycloak%20for%20Authentication.png>

<https://xescuder.medium.com/crear-una-clase-python-y-su-tabla-relacional-con-sqlalchemy-y-alembic-46547a3d56c9>

<https://aws.amazon.com/es/what-is/architecture-diagramming/>